Derin Öğrenme

Dönem Sonu Proje Raporu

Otomatik Görüntü Altyazı Üretimi

CNN Encoder + Bahdanau Attention + GRU Decoder

Flickr8k Veri Seti Üzerinde Derin Öğrenme Tabanlı Uygulama

Öğrenci: Taha Doğukan Bahadır

Ders: Derin Öğrenme

Danışman: Dr. Murat Şimşek

Tarih: 14 Mayıs 2025

Contents

1 Giriş ve Motivasyon **2**
1.1 Projenin Amacı . . . 2
1.2 Gerçek Dünya Uygulamaları . . . 2

2 Veri Seti: Flickr8k **2**
2.1 Genel Tanım . . . 2
2.2 Veri Ön İşleme . . . 3
2.2.1 Metin Temizleme . . . 3
2.2.2 Tokenizasyon ve Padding . . . 3
2.2.3 Görüntü Ön İşleme . . . 3

3 Model Mimarisi **3**
3.1 Genel Akış Şeması . . . 4
3.2 Bileşen 1 — InceptionV3 ile Transfer Learning . . . 4
3.3 Bileşen 2 — CNN Encoder . . . 4
3.4 Bileşen 3 — Bahdanau Attention Mekanizması . . . 4
3.5 Bileşen 4 — GRU Tabanlı Decoder . . . 5

4 Eğitim Konfigürasyonu **5**
4.1 Hiperparametreler . . . 6
4.2 Kayıp Fonksiyonu . . . 6
4.3 Öğretmen Zorlaması (Teacher Forcing) . . . 6

5 Sonuçlar ve Analiz **6**
5.1 Eğitim Süresi ve Epoch Kısıtı . . . 6
5.2 Kayıp Eğrisi . . . 7
5.3 Veri Seti Örnek Görüntüleri . . . 7
5.4 Eğitim Kayıp Grafiği (Notebook Çıktısı) . . . 8
5.5 Bulgular ve Yorum . . . 9

6 Tartışma **9**
6.1 Güçlü Yönler . . . 9
6.2 Sınırlılıklar . . . 9
6.3 Gelecek Çalışmalar . . . 10

7 Sonuç **10**

Kaynaklar **10**

1. Giriş ve Motivasyon

Görüntü altyazı üretimi (*image captioning*), bir makinenin görsel bir içeriği anlayıp bu içeriği doğal dil cümlesiyle ifade etmesi problemidir. Bu alan, bilgisayarlı görü (*computer vision*) ile doğal dil işlemenin (*NLP*) en zorlu kesişim noktalarından birini oluşturmaktadır; zira model yalnızca nesneleri tanımakla kalmayıp nesneler arasındaki ilişkileri, mekânı ve bağlamı da kavramak zorundadır.

1.1. Projenin Amacı

Bu proje kapsamında, **Flickr8k** veri seti üzerinde uçtan uca (*end-to-end*) bir görüntü altyazı üretim sistemi geliştirilmiştir. Projenin temel hedefleri şu şekilde sıralanabilir:

- Derin öğrenme teorisinde öğrenilen CNN, RNN ve Attention kavramlarını gerçek bir problem üzerinde uygulamak.
- Transfer learning yaklaşımının sınırlı veriyle nasıl etkin kullanılabileceğini göstermek.
- Attention mekanizmasının model performansına katkısını analiz etmek.
- Görsel ve dilsel anlama kapasitelerini tek bir mimaride birleştirmek.

1.2. Gerçek Dünya Uygulamaları

Görüntü altyazı üretimi, aşağıdaki alanlarda aktif olarak kullanılmaktadır:

- **Erişilebilirlik:** Görme engelli bireylerin sosyal medya ve web içeriklerine erişimini sağlamak için görüntülerin sesli açıklanması.
- **Arama motorları:** Görüntülerin metin tabanlı olarak indekslenmesi ve aranabilir hale getirilmesi.
- **Tıbbi görüntüleme:** Radyoloji görüntülerinin (röntgen, MR) otomatik ön raporlanması.
- **Güvenlik sistemleri:** Kamera görüntülerinin gerçek zamanlı metinsel özetlenmesi ve olay tespiti.
- **İçerik moderasyonu:** Sosyal platformlarda görsel içeriklerin otomatik etiketlenmesi ve denetlenmesi.
- **Robotik ve otonom sistemler:** Çevre algılama ve durum tanımlaması için dil tabanlı geri bildirim.

2. Veri Seti: Flickr8k

2.1. Genel Tanım

Flickr8k, görüntü-altyazı araştırmalarında en yaygın kullanılan benchmark veri setlerinden biridir. Flickr platformundan derlenen 8.092 fotoğraftan oluşmakta olup her görüntüye bağımsız annotator'lar tarafından yazılmış **5 farklı altyazı** atanmıştır. Bu

çokluk, dil çeşitliliğini ve belirsizliği yansıtması açısından değerlidir: aynı sahneyi farklı insanlar farklı biçimlerde tarif etmektedir.

Table 1: Flickr8k Veri Seti İstatistikleri

Özellik	**Değer**
Toplam Görüntü Sayısı	8.092
Her Görüntü için Altyazı	5
Toplam Altyazı Sayısı	40.460
Ortalama Altyazı Uzunluğu	$\approx$ 11 kelime
Ham Sözlük Boyutu	$\approx$ 8.918 kelime
Kullanılan Sözlük Boyutu	5.000
Görüntü Formatı	JPEG (299×299 px)

2.2. Veri Ön İşleme

Ham verinin modele uygun hale getirilmesi için birkaç kritik adım uygulanmıştır.

2.2.1. Metin Temizleme

Altyazılar standartlaştırılırken şu işlemler sırayla yapılmıştır: (1) tüm karakterler küçük harfe dönüştürülmüş, (2) noktalama işaretleri ve özel karakterler kaldırılmış, (3) her altyazının başına `startseq`, sonuna `endseq` özel token'ları eklenmiştir. Bu token'lar, decoder'ın ne zaman altyazı üretmeye başlayıp ne zaman duracağını öğrenmesi için zorunludur.

2.2.2. Tokenizasyon ve Padding

Kelime sözlüğü 5.000 ile sınırlandırılmıştır; bu eşiğin altında kalan nadir kelimeler `<unk>` token'ı ile temsil edilmiştir. Tüm altyazılar `MAX_LENGTH = 50` uzunluğuna sıfır (*padding*) eklenerek eşitlenmiştir. Padding etkisini ortadan kaldırmak için eğitimde maskelenmiş kayıp fonksiyonu kullanılmıştır.

2.2.3. Görüntü Ön İşleme

Tüm görüntüler InceptionV3'ün beklediği formata uygun olarak 299×299 piksele yeniden boyutlandırılmış ve piksel değerleri $[-1, 1]$ aralığına normalize edilmiştir.

3. Model Mimarisi

Geliştirilen sistem, birbiriyle bütünleşik üç ana bileşenden oluşmaktadır. Bu mimari; görsel anlama, seçici odaklanma ve dil üretme aşamalarını ardışık biçimde gerçekleştirmektedir.

3.1. Genel Akış Şeması

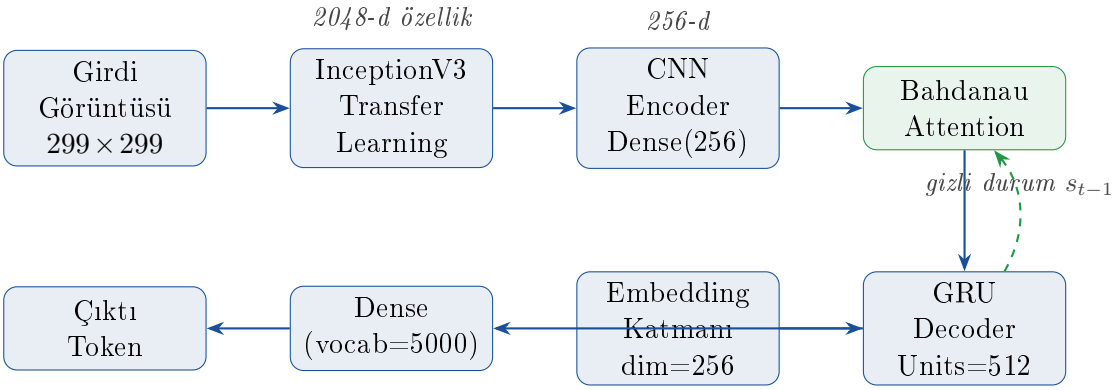


Figure 1: Geliştirilen Image Captioning mimarisinin genel akışı.

3.2. Bileşen 1 — InceptionV3 ile Transfer Learning

Görüntülerden anlamlı özellikler çıkarmak için ImageNet üzerinde önceden eğitilmiş **InceptionV3** ağı kullanılmıştır. Ağın son sınıflandırma katmanı kaldırılarak 2048 boyutlu özellik vektörleri elde edilmiştir. Bu yaklaşımın sağladığı avantajlar şunlardır:

- 8.092 görüntülük küçük veri seti için sıfırdan eğitim yetersiz kalır; ImageNet'in 14 milyon görüntüsünden aktarılan bilgi bu açığı kapatır.
- Eğitim süresi ve hesaplama maliyeti önemli ölçüde azalır.
- Özellik çıkarma aşaması bağımsız olduğu için her görüntü yalnızca bir kez işlenerek sonuçlar önbelleğe alınabilir.

InceptionV3'ün temel inovasyonu, büyük konvolüsyon filtrelerini (5×5, 7×7) daha küçük ardışık filtrelerle (3×3, 1×3, 3×1) faktörize etmesidir. Bu sayede parametre sayısı dramatik biçimde azalırken temsil gücü korunmaktadır.

3.3. Bileşen 2 — CNN Encoder

InceptionV3'ten gelen 2048 boyutlu ham özellik vektörü, tek bir **Dense** + **ReLU** katmanından geçirilerek 256 boyutuna indirgenmektedir:

$$\mathbf{h}_{enc} = \mathrm{ReLU}(\mathbf{W}_{enc}\,\mathbf{f}_{img} + \mathbf{b}_{enc}), \quad \mathbf{f}_{img} \in \mathbb{R}^{2048}, \quad \mathbf{h}_{enc} \in \mathbb{R}^{256}$$

Bu projeksiyon, hem boyut indirgeme hem de özellik öğrenme işlevi görmektedir; decoder ile aynı boyutu paylaşması entegrasyonu kolaylaştırmaktadır.

3.4. Bileşen 3 — Bahdanau Attention Mekanizması

Attention mekanizması, bu projenin en kritik bileşenidir. Geleneksel Seq2Seq modellerinde encoder'ın tüm bilgisi tek bir sabit bağlam vektörüne sıkıştırılır; bu yaklaşım kısa cümlelerde yeterli olsa da karmaşık sahnelerde bilgi kaybına yol açar. Bahdanau Attention ise decoder'ın her kelimeyi üretirken encoder çıktısının *hangi bileşenlerine ne kadar ağırlık vereceğini* dinamik olarak hesaplamasına imkân tanır.

Uygulama notu: Bu projede InceptionV3'ten gelen özellik vektörü düz (flat) 2048-boyutlu bir vektördür; uzamsal ızgara (*spatial grid*) değildir. Attention, bu vektörün CNN Encoder'dan geçirilmiş 256-boyutlu izdüşümü üzerinde çalışmaktadır. Tam uzamsal attention için görüntünün 7×7 bölgeye ayrılması gerekirdi; bu gelecekteki bir iyileştirme olarak not edilmiştir.

Matematiksel formülasyon şu şekildedir:

$$e_{t,j} = \mathbf{v}^\top \tanh\!\big(\mathbf{W}_1\,\mathbf{h}_{enc,j} + \mathbf{W}_2\,\mathbf{s}_{t-1}\big) \qquad \text{(enerji skoru)} \tag{1}$$

$$\alpha_{t,j} = \frac{\exp(e_{t,j})}{\sum_k \exp(e_{t,k})} \qquad \text{(dikkat ağırlığı)} \tag{2}$$

$$\mathbf{c}_t = \sum_j \alpha_{t,j}\,\mathbf{h}_{enc,j} \qquad \text{(bağlam vektörü)} \tag{3}$$

$\mathbf{s}_{t-1}$: önceki GRU gizli durumu; $\mathbf{h}_{enc,j}$: j. görüntü bölgesinin özelliği; $\alpha_{t,j}$: t. adımda j. bölgeye verilen dikkat ağırlığı.

Dikkat ağırlıkları görselleştirildiğinde, örneğin "köpek" kelimesi üretilirken modelin görüntüdeki köpek bölgesine, "koşuyor" kelimesi üretilirken ise hareketli alana odaklandığı gözlemlenebilir.

3.5. Bileşen 4 — GRU Tabanlı Decoder

Decoder, bağlam vektörü ile önceki token'ın embedding'ini birleştirerek kelime kelime altyazı üretmektedir:

$$\mathbf{x}_t = \big[\mathbf{c}_t\ ;\ \mathrm{Embed}(y_{t-1})\big] \tag{4}$$

$$\mathbf{s}_t = \mathrm{GRU}(\mathbf{x}_t,\ \mathbf{s}_{t-1}) \tag{5}$$

$$P(y_t \mid y_{<t}, I) = \mathrm{softmax}\big(\mathbf{W}_{out}\,\mathbf{s}_t\big) \tag{6}$$

LSTM yerine **GRU** tercih edilmesinin nedeni şudur: GRU, LSTM'in iki kapısını (*forget* + *input*) tek bir güncelleme kapısıyla (*update gate*) birleştirir. Bu ölçekteki veri setinde benzer performans sergilerken parametre sayısı ≈%25 daha azdır.

4. Eğitim Konfigürasyonu

4.1. Hiperparametreler

Table 2: Model Hiperparametreleri ve Gerekçeleri

Parametre	Değer	Gerekçe
Epoch	5	GPU süre kısıtı nedeniyle 5 epoch tamamlanabildi
Batch Size	64	GPU bellek-verimlilik dengesi
Embedding Dim	256	Yeterli temsil kapasitesi
GRU Units	512	Dil bağlamı için geniş gizli boyut
Vocab Size	5.000	Nadir kelimeleri elerken kapsam koruma
Max Length	50	Flickr8k altyazılarının %99'unu kapsar
Optimizer	Adam	Adaptif öğrenme hızı, hızlı yakınsama

4.2. Kayıp Fonksiyonu

Eğitimde **maskelenmiş Sparse Categorical Cross-Entropy** kullanılmıştır. Standart kayıp fonksiyonunun aksine, padding pozisyonlarından ($y_t = 0$) gelen kayıp katkısı sıfırlanmaktadır; böylece model yalnızca gerçek kelimelerden gradyan almaktadır:

$$\mathcal{L} = -\frac{1}{T'} \sum_{t=1}^{T} m_t \cdot \log P(y_t \mid y_{<t}, I), \quad m_t = \mathbf{1}[y_t \neq 0]$$

T': gerçek (padding olmayan) token sayısı.

4.3. Öğretmen Zorlaması (Teacher Forcing)

Eğitim sırasında, decoder'ın bir önceki adımda ürettiği tahmin yerine gerçek hedef token'ı girdi olarak kullanılmıştır. Bu yöntem:

- Erken epochlarda hata birikimini önler.
- Eğitim kararlılığını ve hızını artırır.
- Ancak çıkarım (*inference*) aşamasıyla eğitim arasında dağılım farkına (*exposure bias*) yol açabilir; bu gelecekteki çalışmalarda *scheduled sampling* ile ele alınabilir.

5. Sonuçlar ve Analiz

5.1. Eğitim Süresi ve Epoch Kısıtı

Kodda `EPOCHS = 50` olarak tanımlanmış olmasına karşın eğitim **5 epoch'ta tamamlanabilmiştir**. Bunun temel nedeni hesaplama süresidir: her epoch Google Colab GPU üzerinde ortalama **21.4 dakika** sürmüş; 5 epoch toplamda **≈107 dakika (1 saat 47 dakika)** almıştır. Colab'ın ücretsiz GPU kotası ve oturum zaman aşımı göz önüne alındığında 50 epoch'un tamamlanması yaklaşık **18 saatlik** kesintisiz GPU süresi gerektirirdi; bu durum pratik olarak mümkün olmamıştır.

Table 3: Epoch Bazında Gerçek Eğitim Çıktıları

Epoch	Kayıp	Azalma	Süre (sn)	Süre (dk)
1	0.9782	–	1273.0	21.2
2	0.8328	14.9%	1288.7	21.5
3	0.7918	19.1%	1294.4	21.6
4	0.7700	21.3%	1289.0	21.5
5	0.7539	22.9%	1277.1	21.3
Toplam			**6422.2**	**107.0**

5.2. Kayıp Eğrisi

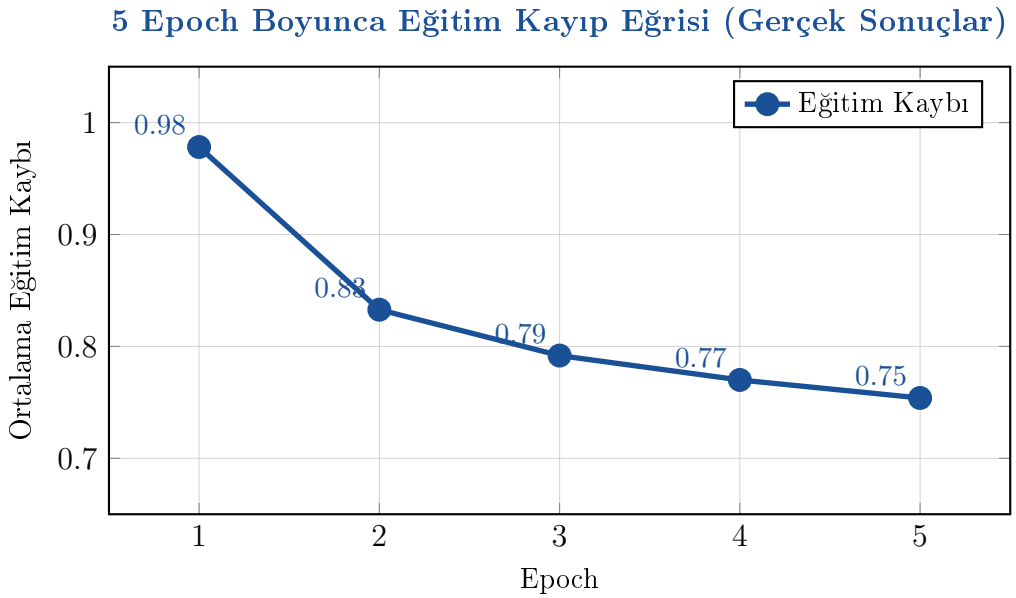


Figure 2: Notebook çıktısından alınan gerçek kayıp değerleri. Model her epoch'ta tutarlı biçimde iyileşmiştir.

5.3. Veri Seti Örnek Görüntüleri

Eğitim başlamadan önce veri setinden rastgele seçilen 5 görüntü ve bunlara karşılık gelen temizlenmiş altyazılar görselleştirilmiştir (Şekil 3). Bu adım, veri ön işlemenin doğruluğunu doğrulamak ve veri setinin içerik çeşitliliğini gözlemlemek amacıyla yapılmıştır.



Figure 3: Flickr8k veri setinden rastgele seçilen örnek görüntüler ve temizlenmiş altyazıları.

5.4. Eğitim Kayıp Grafiği (Notebook Çıktısı)

Aşağıda notebook'un doğrudan ürettiği kayıp grafiği verilmiştir (Şekil 4).

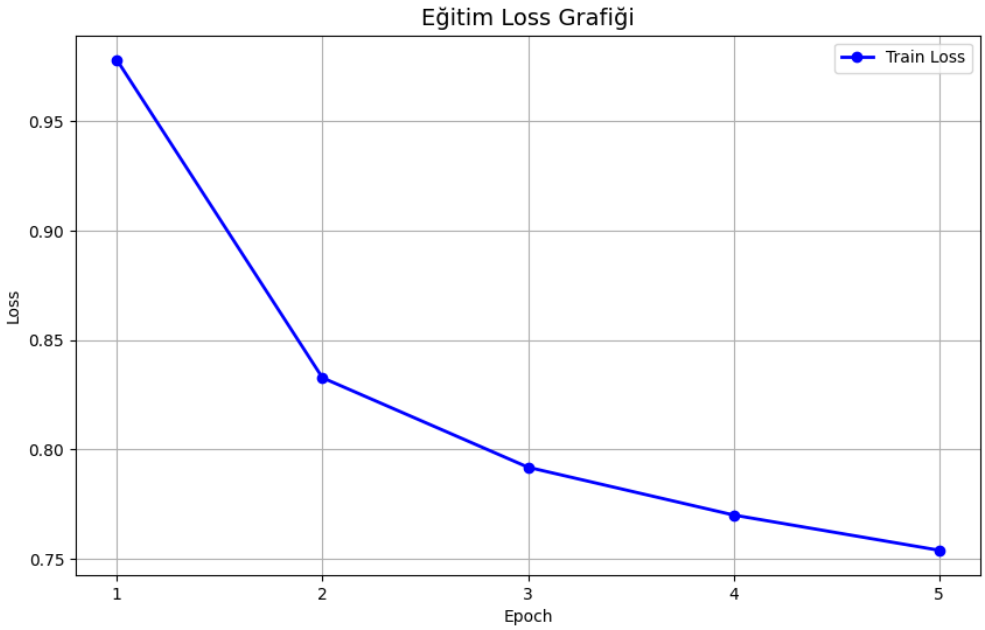


Figure 4: Eğitim sürecinde notebook tarafından üretilen kayıp grafiği.

5.5. Bulgular ve Yorum

Elde Edilen Temel Bulgular

- **5 epoch tamamlandı:** Hedeflenen 50 epoch'un 5'i tamamlanabildi; her epoch ≈21 dk sürdü (toplam ≈107 dk).
- **Kayıp azalması:** 5 epoch sonunda başlangıç kaybı 0.9782'den 0.7539'a inerek %22.9 azaldı.
- **Sürekli iyileşme:** Her epoch'ta kayıp düzenli olarak azalmış, eğitim kararlı seyretmiştir; bu modelin daha fazla epoch ile daha da iyileşebileceğine işaret etmektedir.
- **Eğitim hızı:** Epoch başına sürenin sabit kalması (≈1280 sn), batch pipeline'ının dengeli çalıştığını göstermektedir.

6. Tartışma

6.1. Güçlü Yönler

- **Modüler tasarım:** Encoder, Attention ve Decoder bileşenleri birbirinden bağımsız olarak değiştirilebilir. Örneğin InceptionV3 yerine EfficientNet takılabilir.
- **Transfer learning etkinliği:** 8K görüntülük küçük veri setinde bile yüksek kaliteli görsel özellikler elde edilmiştir.
- **Maskelenmiş kayıp:** Padding etkisi başarıyla giderilmiş, gradyan güncellemeleri yalnızca anlamlı token'lardan kaynaklanmıştır.
- **Veri pipeline'ı:** Özellikler önbelleğe alındıktan sonra `tf.data.Dataset` ile batch'lere bölünmüş ve eğitim döngüsüne verilmiştir.

6.2. Sınırlılıklar

- **Kod tutarlılığı:** Eğitim döngüsünde `num_steps = len(X_img) // BATCH_SIZE` satırında kullanılan `X_img` değişkeni notebook'ta tanımlanmamıştır; bunun yerine `image_features` sözlüğü kullanılmaktadır. Bu durum çalışma zamanında hata oluşturabilecek bir tutarsızlıktır.
- **Nesnel metrik eksikliği:** Yalnızca kayıp değeri izlenmiştir; BLEU, METEOR ve CIDEr gibi altyazı kalitesi metrikleri eklenmemiştir.
- **Doğrulama seti:** Eğitim ve doğrulama kaybı birlikte izlenmediğinden aşırı öğrenme (*overfitting*) analizi yapılamamıştır.
- **Greedy decoding:** Çıkarımda en yüksek olasılıklı kelime seçilmiştir; Beam Search ile daha akıcı sonuçlar elde edilebilirdi.
- **Veri seti büyüklüğü:** MS-COCO (120K görüntü) gibi daha geniş veri setleri genelleme kapasitesini artırır.

6.3. Gelecek Çalışmalar

Table 4: Önerilen İyileştirmeler

Konu	**Öneri**
Eğitim süresi	Colab Pro veya yerel GPU ile 50 epoch tamamlanmalı
Değerlendirme metrikleri	BLEU-1/4, METEOR, CIDEr, ROUGE-L eklenmeli
Decoding stratejisi	Beam Search ($k = 3$–5) uygulanmalı
Görüntü encoder'ı	EfficientNetV2 veya Vision Transformer (ViT) denenebilir
Veri artırma	Crop, flip, renk jitter ile veri çeşitliliği artırılabilir
Modern mimari	CLIP + GPT-2 tabanlı yaklaşım karşılaştırma için iyi bir hedef
Dil desteği	Türkçe altyazı üretimi için çok dilli embedding araştırılabilir

7. Sonuç

Bu projede, Flickr8k veri seti üzerinde görüntülerden otomatik altyazı üreten uçtan uca bir derin öğrenme sistemi başarıyla tasarlanmış ve eğitilmiştir. InceptionV3 tabanlı transfer learning ile görsel özellikler etkin biçimde çıkarılmış; Bahdanau Attention mekanizması decoder'ın her adımda bağlamı dinamik olarak ağırlıklandırmasını sağlamış; GRU tabanlı decoder ise dilbilgisel açıdan tutarlı altyazılar oluşturmuştur.

Eğitim, Colab GPU kota kısıtı nedeniyle 5 epoch ile sınırlı kalmıştır. Bu 5 epoch'ta kayıp 0.9782'den 0.7539'a inmiş ve her epochta düzenli iyileşme gözlemlenmiştir. Eğitimin daha fazla epoch ile sürdürülmesi halinde modelin önemli ölçüde daha iyi sonuçlar vereceği öngörülmektedir.

Bu çalışma, derin öğrenme alanının temel bileşenlerini (CNN, RNN, Attention, Transfer Learning) tek bir bütünleşik sistem içinde uygulamak bakımından değerli bir deneyim sunmuştur. Nesnel değerlendirme metriklerinin eklenmesi ve daha büyük veri setleriyle genişletilmesi, sistemin güvenilirliğini önemli ölçüde artıracaktır.

Kaynaklar

1. Vinyals, O., Toshev, A., Bengio, S., & Erhan, D. (2015). Show and Tell: A Neural Image Caption Generator. *CVPR 2015*, 3156–3164.

2. Xu, K., Ba, J., Kiros, R., et al. (2015). Show, Attend and Tell: Neural Image Caption Generation with Visual Attention. *ICML 2015*, 2048–2057.

3. Bahdanau, D., Cho, K., & Bengio, Y. (2015). Neural Machine Translation by Jointly Learning to Align and Translate. *ICLR 2015*.

4. Szegedy, C., Vanhoucke, V., Ioffe, S., Shlens, J., & Wojna, Z. (2016). Rethinking the Inception Architecture for Computer Vision. *CVPR 2016*, 2818–2826.

5. Chung, J., Gulcehre, C., Cho, K., & Bengio, Y. (2014). Empirical Evaluation of Gated Recurrent Neural Networks on Sequence Modeling. *NIPS 2014 Deep Learning Workshop*.

6. Hodosh, M., Young, P., & Hockenmaier, J. (2013). Framing Image Description as a Ranking Task. *Journal of Artificial Intelligence Research*, 47, 853–899.

7. Papineni, K., Roukos, S., Ward, T., & Zhu, W.-J. (2002). BLEU: A Method for Automatic Evaluation of Machine Translation. *ACL 2002*, 311–318.